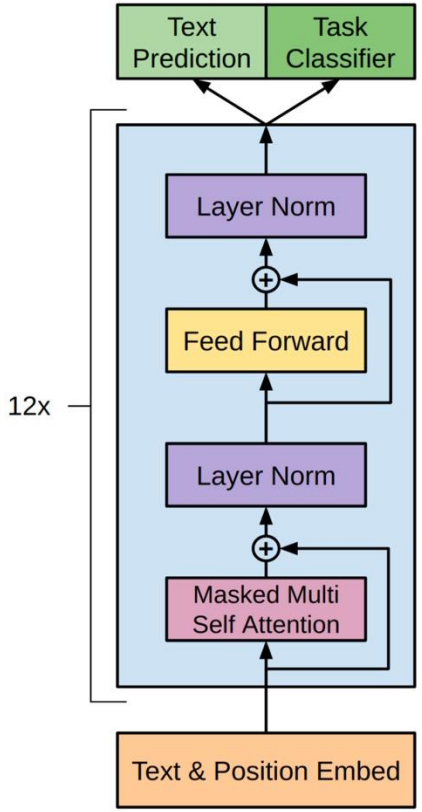
ECE 590
Neural-Network-Based Large Language Models

Tuesdays and Thursdays 1:25-2:40
Fitzpatrick Schiciano A 1464

ECE 590
Neural-Network-Based Large Language Models

Lecture 14
03/03/2026

Sequence-to-Sequence (seq2seq) Language Models



Prompt/question

Retrieved text

$$p_\theta(y_i | x, z, y_{1:i-1})$$

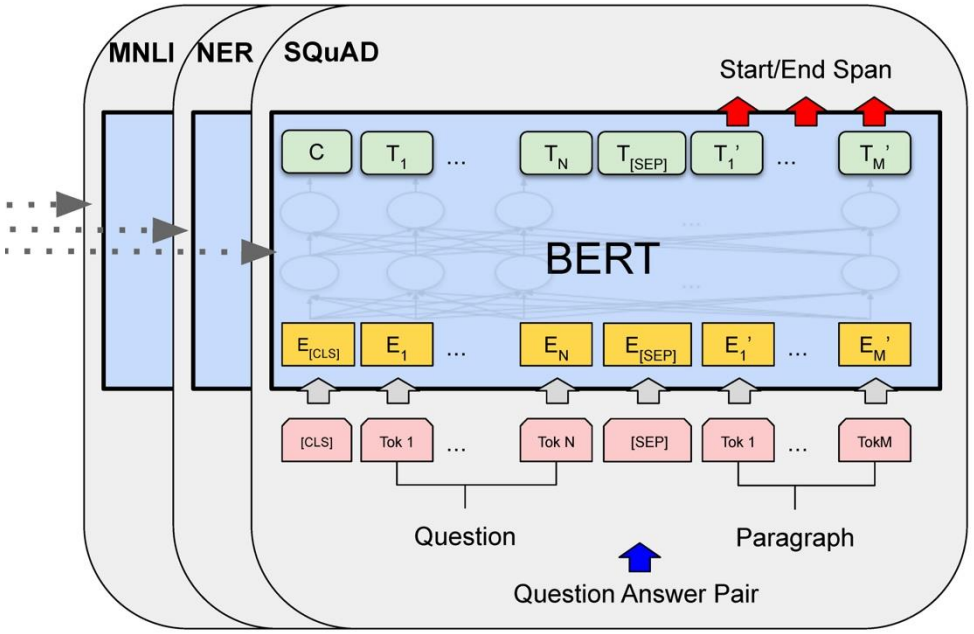
Next token

Text generated thus far

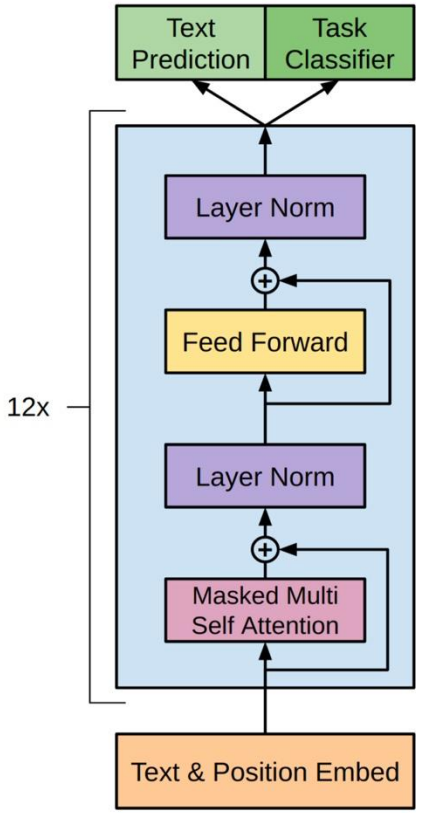
Two Models for RAG

- BERT for retrieval and GPT for seq2seq generation

- Fine-tune both together for RAG application



Fine-Tuning



Sequence Model

- Given input x and retrieved text z the probability of the generating sequence y, composed of N tokens is

$$p(y|x,z) = \prod_{i=1}^{N} P_\theta(y_i|x,z,y_{1:i-1})$$

- While we may understand this probabilistic equation, how do we actually generate the sequence y?

- Has N output tokens

- We have a softmax probability at each step

Sequence Model

- Given input x and retrieved text z the probability of the generating sequence y, composed of N tokens is

$$p(y|x,z) = \prod_{i=1}^{N} P_\theta(y_i|x,z,y_{1:i-1})$$

- While we may understand this probabilistic equation, how do we actually generate the sequence y?

- Has N output tokens

- We have a softmax probability at each step; could just pick the most probable word at each step

- Generally does not generate particularly realistic text: Beam search

BERT-Based Selection from Database of Documents

- Consider prompt x and element from database z

- Embed prompt and database element via BERT (using output above [CLS]):

$$d(z) = \mathrm{BERT}_d(z) \in R^M \qquad\qquad q(x) = \mathrm{BERT}_q(x) \in R^M$$

- The probability of z matching x satisfies

$$p_\eta(z|x) \propto \exp[d(z) \cdot q(x)/\eta]$$

- In general, we train two different BERT models, one for question and the other for the database

BERT-Based Selection from Database of Documents

$$p_\eta(z|x) \propto \exp[d(z) \cdot q(x)/\eta]$$

- Would be standard softmax if we computed this probability using all elements in the database
- We only consider the K most probable elements in the database, and normalize accordingly
- Let $S_q(x)$ represent the set of K most probable elements from the database for prompt x

$$p_\eta\big(z \in S_q(x)\big|x\big) = \frac{\exp[d\big(z \in S_q(x)\big) \cdot q(x)/\eta]}{\sum_{z' \in S_q(x)} \exp[d(z')) \cdot q(x)/\eta]}$$

- Sum involves K elements $z' \in S_q(x)$

In Practice, Two Approaches for RAG Implementation

- **RAG sequence**:
  - For text z retrieved from the database, generate text

$$p(y|z,x) = \prod_{i=1}^{n} p(y_i|z,x,y_{1:i-1})$$

  - For each pair (x,z) generate candidate text y, and select the most probable set of possible generated text via beam search
  - Do this for all candidate text from database, $z \in S_{q(x)}$
  - For all candidate generated text, compute

$$p(y|x) = \sum_{z \in S_q(x)} p_\eta(z|x)p(y|x,z)$$

In Practice, Two Approaches for RAG Implementation

- **RAG token**:
  - With RAG-Sequence, we average over $z \in S_{q(x)}$ *after* generating all possible complete generated text
  - With RAG-Token, we average over $z \in S_{q(x)}$ when selecting each token, based on text $y_{1:i-1}$ generated thus far

$$p(y_i|x, y_{1:i-1}) = \sum_{z \in S_{q(x)}} p_\gamma(z|x) p(x, z, y_{1:i-1})$$

  - All candidate generated at once by beam search, token by token, averaging over $p_\gamma(z|x)$ at each step
  - Same set of prior generations $y_{1:i-1}$ used for all $z \in S_{q(x)}$

Open Questions

- GPT and BERT style Transformers are powerful models
- For text generation, GPT class models are necessary
- Challenge: With cross-entropy loss, we train them to predict next word
- If we generate text by taking the most probable output at each step, text is typically not natural
- Beam search is a significant improvement for generation of realistic text
- Still: The model is not trained such that it is aligned with human preferences; no reason to believe that cross entropy loss with one word at a time will yield text aligned with human preferences, even with beam search

Leveraging Human Feedback

- Modern Transformers give opportunity to provide feedback
- Thumbs up/thumbs down
- People also often write ”thank you” when conversing with an LLM
- One can also examine the human interaction to detect dissatisfaction with the generated text
- Here we assess the entire generated text, not just one word at a time
- How do we develop a loss function that accounts for human feedback on entire text?

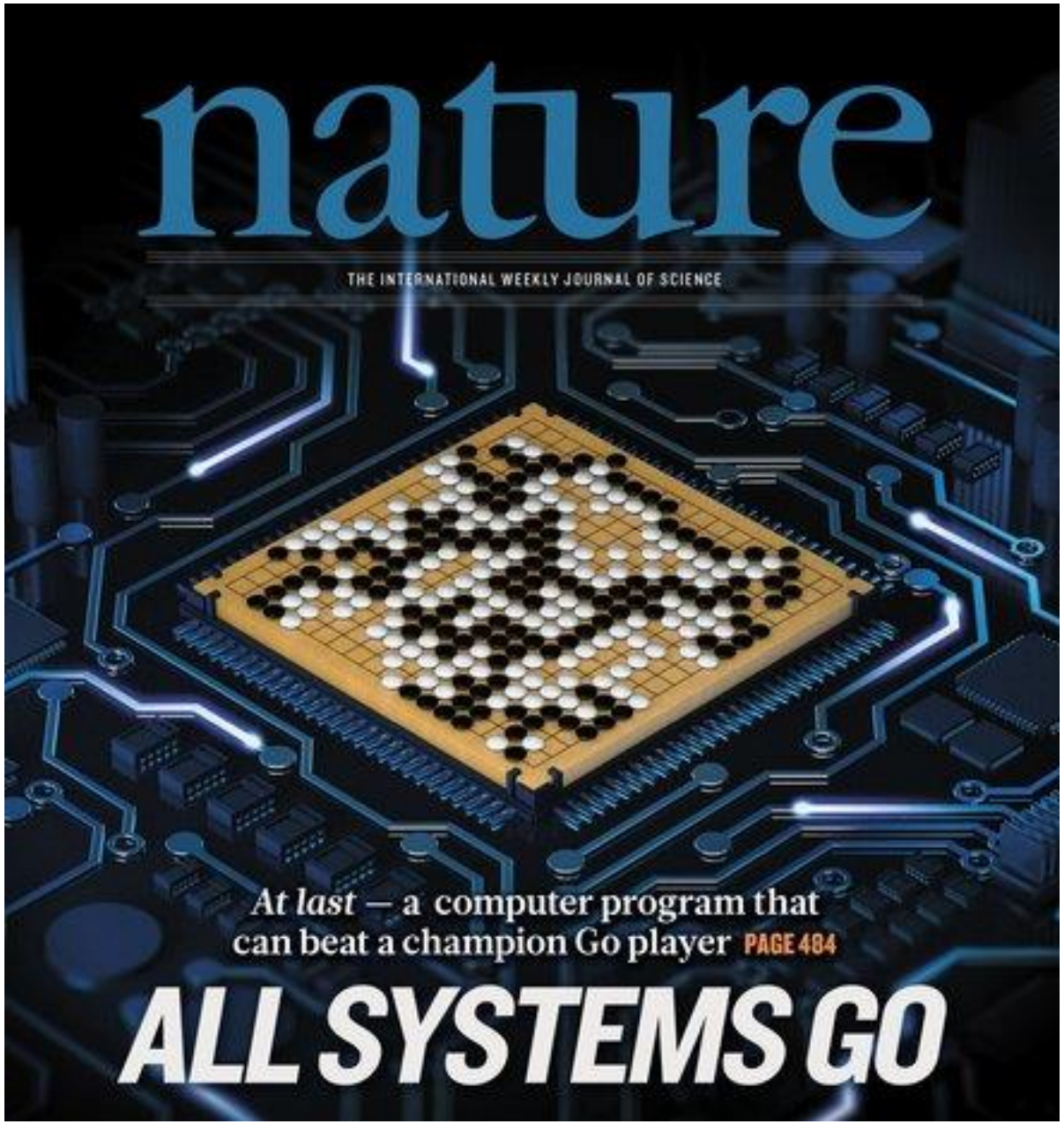
Alignment with Reinforcement Learning

- Reinforcement learning is like learning how to ride a bike just based on feedback on how you have done so far
- If you do some actions that keep the bike moving, and you atop it: Do more of that.
- If you do something that leads to an abrupt fall: Don’t do that again.
- Through continual trial and error, based on feedback, you learn to ride a bike, with no formal instructions
- This is reinforcement learning; reinforce actions that yield high rewards, based on feedback
- Use human feedback on LLM outputs, and based on that refine and continually improve the model

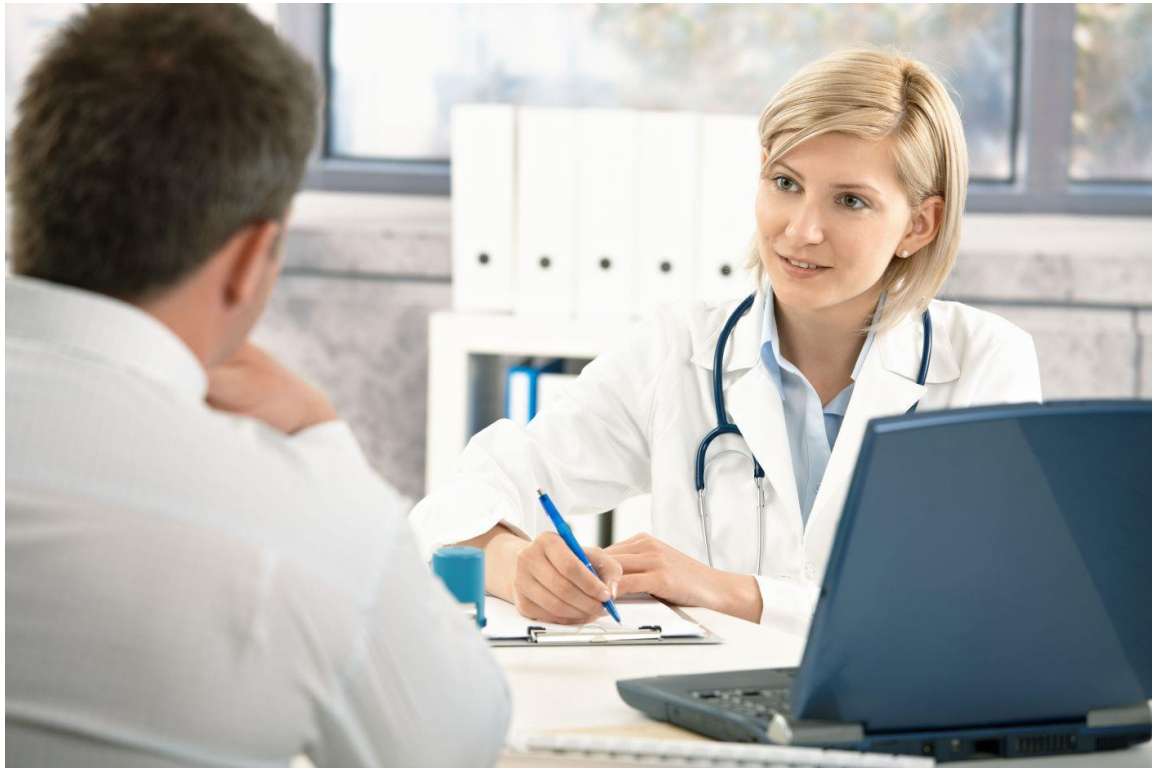
Reinforcement Learning

- Reinforcement learning is a field of long-term study in machine learning
- Build on that, and tailor it to the special characteristics of language
- In addition to being of value for improving LLMs, the study of RL is of general interest in machine learning
- We will next study RL in the context of aligning LLM outputs with the desires of humans
- Further, we will examine RL to teach an LLM to perform chain-of-thought reasoning

Reinforcement Learning

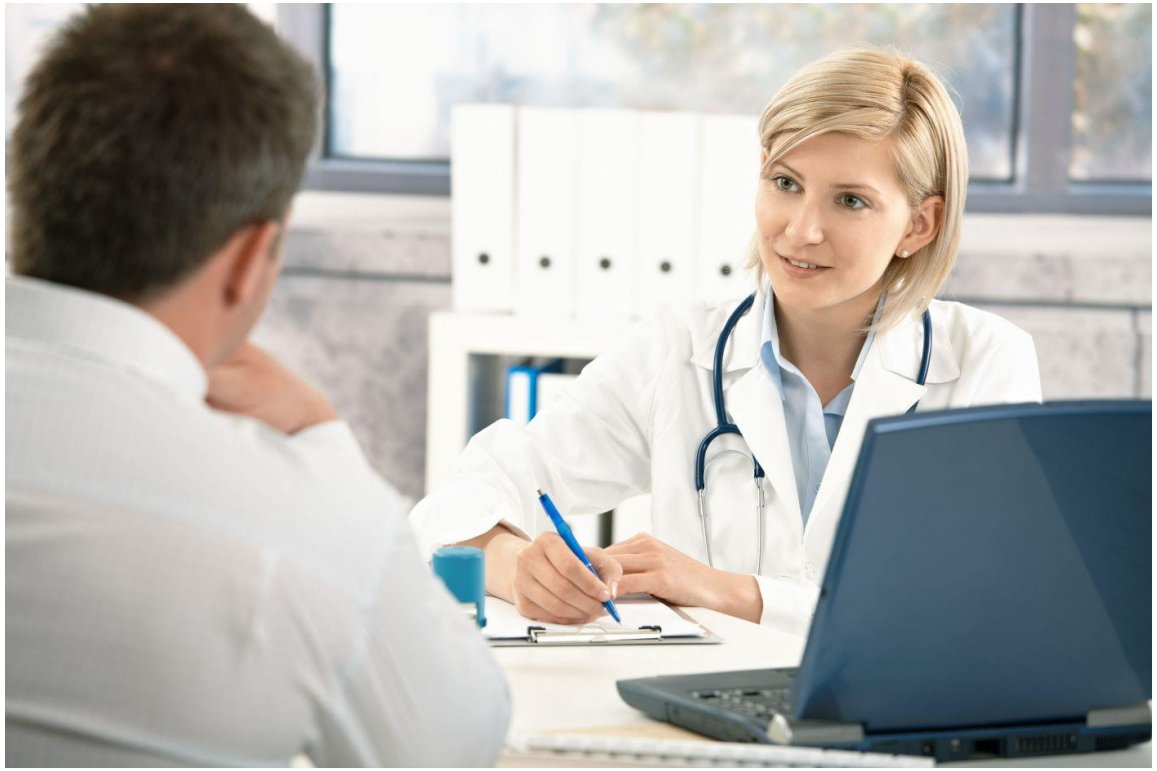

nature
THE INTERNATIONAL WEEKLY JOURNAL OF SCIENCE
At last — a computer program that can beat a champion Go player PAGE 484
ALL SYSTEMS GO

NETFLIX
ALPHAGO

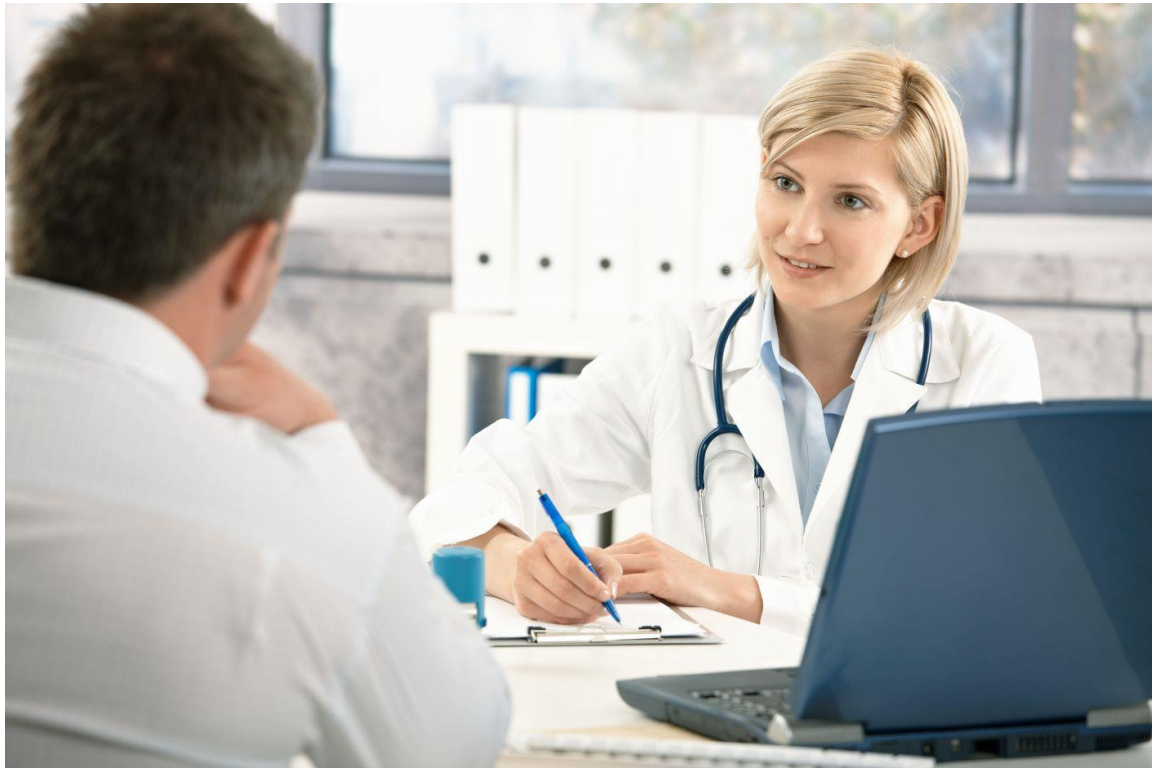


- Consider an MD, seeking to treat patients in an effective manner, while minimizing costs

- The state of health for each patient is observed in terms of a set of clinical variables



- Consider an MD, seeking to treat patients in an effective manner, while minimizing costs
- The state of health for each patient is observed in terms of a set of clinical variables
- Assume there are a set of actions the MD can perform
- Each action will impact the health state of the patient (possibly no change in the state)



- Consider an MD, seeking to treat patients in an effective manner, while minimizing costs
- The state of health for each patient is observed in terms of a set of clinical variables
- Assume there are a set of actions the MD can perform
- Each action will impact the health state of the patient (possibly no change in the state)
- Seek a policy that achieves the best outcomes at the lowest average price
- Could apply to particular types of patients (e.g., diabetics) or in particular settings within a health system (e.g., operating room)

- Assume the set of observations defining the patient **state** represented by vector s

- Assume the set of **actions** that may be taken is $A = \{a_1, \dots, a_m\}$

- Assume the set of observations defining the patient **state** represented by vector s
- Assume the set of **actions** that may be taken is $A = \{a_1, \dots, a_m\}$
- There is **randomness** in how a patient in state s will respond to a given action
- Let $P(s, a, s')$ represent the *probability* that when a patient is in state s and the MD takes action a, the patients state will change to s'

- Assume the set of observations defining the patient **state** represented by vector s
- Assume the set of **actions** that may be taken is $A = \{a_1, \dots, a_m\}$
- There is **randomness** in how a patient in state s will respond to a given action
- Let $P(s, a, s')$ represent the *probability* that when a patient is in state s and the MD takes action a, the patients state will change to s'
- Let $r(s, a, s')$ represent the "**reward**" associated with the MD taking action a for a patient in state s, and then the patient transiting to new state of health s'
- The reward $r(s, a, s')$ may reflect the *immediate* impact for the patient of the change $s \rightarrow s'$ and also the cost of action a

- The MD interacts with the patient through a series of actions, patient state changes, and rewards/costs

$$\ldots \quad s_{t-1} \quad a_{t-1} \quad r_{t-1} \quad s_t \quad a_t \quad r_t \quad s_{t+1} \quad \ldots$$

- The MD interacts with the patient through a series of actions, patient state changes, and rewards/costs

$$\ldots \quad s_{t-1} \quad a_{t-1} \quad r_{t-1} \quad s_t \quad a_t \quad r_t \quad s_{t+1} \ldots$$

- Goal: Develop a **policy** that defines for the MD the optimal action a to take when presented by a patient in state s; the policy may define *standard of care*

- The optimal policy will maximize the average reward over time, with reward accounting for patient outcome and costs

- The MD interacts with the patient through a series of actions, patient state changes, and rewards/costs

$$\ldots \quad s_{t-1} \quad a_{t-1} \quad r_{t-1} \quad s_t \quad a_t \quad r_t \quad s_{t+1} \ldots$$

- Goal: Develop a **policy** that defines for the MD the optimal action a to take when presented by a patient in state s; the policy may define *standard of care*

- The optimal policy will maximize the average reward over time, with reward accounting for patient outcome and costs

- The policy should be non-myopic, in that it thinks ahead, to the long-run impact of actions

- The policy will typically weight impacts in the near-term more highly than what happens in the long run

Big Challenge

- We typically do not know $P(s, a, s')$
- How can we learn a policy without this?
- We can just experience/try things, keep a record of outcomes, and adapt and adjust

Big Challenge

- We typically do not know $P(s, a, s')$
- How can we learn a policy without this?
- We can just experience/try things, keep a record of outcomes, and adapt and adjust
- Reinforce actions for particular patient states that are rewarding
- Discourage actions that are expensive and yield poor outcomes
- In many ways this is how medicine works, over time

- Reinforcement learning is the formalization of this challenge
- Addresses sequential decision making in an uncertain (stochastic) world

Medicine/Health

- Reinforcement learning is the formalization of this challenge

- Addresses sequential decision making in an uncertain (stochastic) world

Medicine/Health

Monitoring/Maintenance of Factory

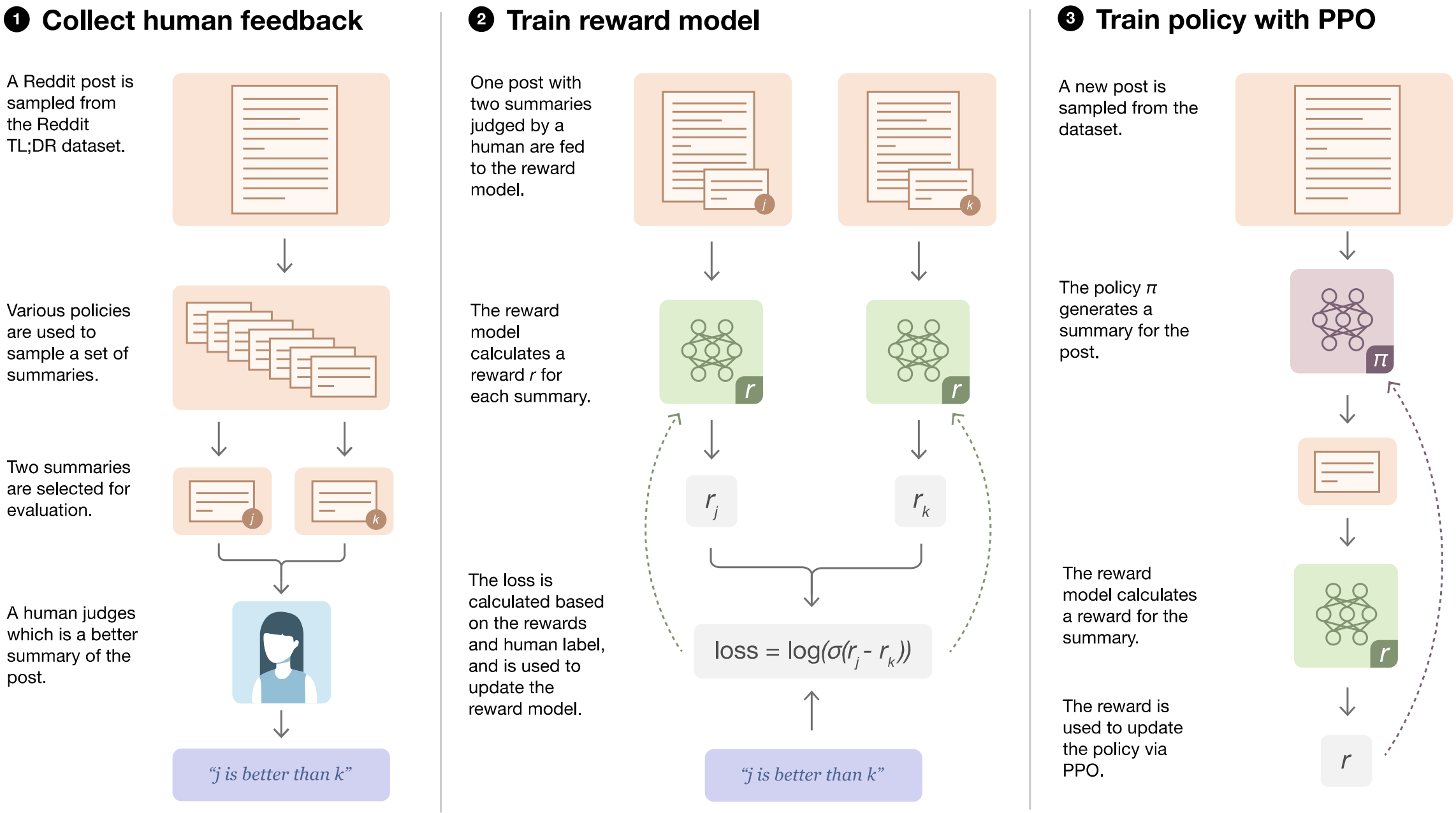
- Reinforcement learning is the formalization of this challenge

- Addresses sequential decision making in an uncertain (stochastic) world

Medicine/Health

Monitoring/Maintenance of Factory

Investing


1 Collect human feedback
A Reddit post is sampled from the Reddit TL;DR dataset.
Various policies are used to sample a set of summaries.
Two summaries are selected for evaluation.
A human judges which is a better summary of the post.
j
k
"j is better than k"
2 Train reward model
One post with two summaries judged by a human are fed to the reward model.
The reward model calculates a reward r for each summary.
The loss is calculated based on the rewards and human label, and is used to update the reward model.
r
r_j
r_k
loss = log(σ(r_j - r_k))
"j is better than k"
3 Train policy with PPO
A new post is sampled from the dataset.
The policy π generates a summary for the post.
The reward model calculates a reward for the summary.
The reward is used to update the policy via PPO.
π
r

Illustrative Example

- Consider diabetes control

- Assume that the patient **state** s is defined by the **minimum and maximum glucose concentration from previous day**

- The **action** a may be the **rate of continuous insulin supply** and the **bolus dose**

Illustrative Example

- Consider diabetes control
- Assume that the patient **state** s is defined by the **minimum and maximum glucose concentration from previous day**
- The **action** a may be the **rate of continuous insulin supply** and the **bolus dose**
- $r(s, a, s')$ may be defined to reward (punish) glucose levels that are desirable (undesirable)
- Assume that we may specify $r(s, a, s')$

Solution Setup

- Assume that the patient **state** s is defined by the **minimum and maximum glucose concentration from previous day**
  - Discretize the continuous range of the state values into n bins
- The **action** a may be the **rate of continuous insulin supply** and the **bolus dose**
  - Discretize the continuous range of the action values into m bins

Solution Setup

- Assume that the patient **state** s is defined by the **minimum and maximum glucose concentration from previous day**
  - Discretize the continuous range of the state values into n bins
- The **action** a may be the **rate of continuous insulin supply** and the **bolus dose**
  - Discretize the continuous range of the action values into m bins
- The Q function $Q(s, a)$ is an $n \times m$ matrix, denoting the value of taking action a when in state s

Simple, Intuitive Solution

- Set initial values of $n \times m$ matrix $Q(s, a)$ based on prior available medical knowledge, or set at random

- After initializing $Q(s, a)$, take an action a when patient is in particular state s, and then observe the new state s'

$$(s, a) \rightarrow s' , \qquad r(s, a, s')$$

Simple, Intuitive Solution

- Set initial values of $n \times m$ matrix $Q(s, a)$ based on prior available medical knowledge, or set at random

- After initializing $Q(s, a)$, take an action a when patient is in particular state s, and then observe the new state s'

$$(s, a) \rightarrow s' , \qquad r(s, a, s')$$

- Assume that the prior/old value for the Q function when taking action a in state s is $Q^{old}(s, a)$

Simple, Intuitive Solution

- Set initial values of $n \times m$ matrix $Q(s,a)$ based on prior available medical knowledge, or set at random

- After initializing $Q(s,a)$, take an action a when patient is in particular state s, and then observe the new state s'

$$(s,a) \rightarrow s' \,, \qquad r(s,a,s')$$

- Assume that the prior/old value for the Q function when taking action a in state s is $Q^{old}(s,a)$

- We may consider the update rule:

$$Q^{new}(s,a) \leftarrow Q^{old}(s,a) + \alpha \cdot [r(s,a,s') - Q^{old}(s,a)], \quad \text{with } \alpha \in (0,1)$$

Simple, Intuitive Solution

$$Q^{new}(s,a) \leftarrow Q^{old}(s,a) + \alpha \cdot \underbrace{[\, r(s,a,s') - Q^{old}(s,a)]}_{\text{Temporal Difference (TD)}}, \quad \text{with } \alpha \in (0,1)$$

- α is called the "learning rate," and controls the relative balance between our old estimate $Q^{old}(s,a)$ and new information provided by $r(s,a,s')$

- If the TD is positive, the inferred value of taking action a in state s is increased; if TD negative, the value of taking action a in state s is diminished

- Very similar to idea we used in gradient descent update of latent function

Simple, Intuitive Solution

$$Q^{new}(s,a) \leftarrow Q^{old}(s,a) + \alpha \cdot [\, r(s,a,s') - Q^{old}(s,a)] \,, \quad \text{with } \alpha \in (0,1)$$

- If the reward $r(s,a,s')$ is large (small) then $Q^{new}(s,a)$ is typically increased (diminished)

Simple, Intuitive Solution

$$Q^{new}(s,a) \leftarrow Q^{old}(s,a) + \alpha \cdot [\, r(s,a,s') - Q^{old}(s,a)] \,, \quad \text{with } \alpha \in (0,1)$$

- If the reward $r(s,a,s')$ is large (small) then $Q^{new}(s,a)$ is typically increased (diminished)
- Problem: This only accounts for the <u>*immediate*</u> reward $r(s,a,s')$
- Doesn't account for what may happen subsequently once the patient state changes to s'

Example: Problem With Myopic Policy

$$Q^{new}(s,a) \leftarrow Q^{old}(s,a) + \alpha \cdot [\, r(s,a,s') - Q^{old}(s,a)] \,, \quad \text{with } \alpha \in (0,1)$$

- Assume state of patient s corresponds to severe poor health

- A particular action a may have probable positive <u>immediate</u> reward $r(s,a,s')$

Example: Problem With Myopic Policy

$$Q^{new}(s,a) \leftarrow Q^{old}(s,a) + \alpha \cdot [\, r(s,a,s') - Q^{old}(s,a)] \,, \quad \text{with } \alpha \in (0,1)$$

- Assume state of patient s corresponds to severe poor health
- A particular action a may have probable positive <u>immediate</u> reward $r(s,a,s')$
- However, there may be serious long-term complications (e.g., a different or more severe future condition)
- A policy that only accounts for the immediate reward would not account for long-term, later consequences

Extension

- Recall our simple solution

$$Q^{new}(s,a) \leftarrow Q^{old}(s,a) + \alpha \cdot [\, r(s,a,s') - Q^{old}(s,a)] \,, \quad \text{with } \alpha \in (0,1)$$

Extension

- Recall our simple solution

$$Q^{new}(s,a) \leftarrow Q^{old}(s,a) + \alpha \cdot [\, r(s,a,s') - Q^{old}(s,a)] \,, \quad \text{with } \alpha \in (0,1)$$

- Consider the extension

$$Q^{new}(s,a) \leftarrow Q^{old}(s,a) + \alpha \cdot [\, r(s,a,s') + \gamma \cdot \max_{a'} Q^{old}(s',a') - Q^{old}\,(s,a)]$$

Extension

- Recall our simple solution

$$Q^{new}(s,a) \leftarrow Q^{old}(s,a) + \alpha \cdot [\, r(s,a,s') - Q^{old}(s,a)] \,, \quad \text{with } \alpha \in (0,1)$$

- Consider the extension

$$Q^{new}(s,a) \leftarrow Q^{old}(s,a) + \alpha \cdot [\, r(s,a,s') + \gamma \cdot \max_{a'} Q^{old}(s',a') - Q^{old}\,(s,a)]$$

Extension

- Recall our simple solution

$$Q^{new}(s,a) \leftarrow Q^{old}(s,a) + \alpha \cdot [\, r(s,a,s') - Q^{old}(s,a)] \,, \quad \text{with } \alpha \in (0,1)$$

- Consider the extension

Immediate Reward

Discount Factor Between 0 and 1

$$Q^{new}(s,a) \leftarrow Q^{old}(s,a) + \alpha \cdot [\, r(s,a,s') + \gamma \cdot \max_{a'} Q^{old}(s',a') - Q^{old}\,(s,a)]$$

Expected Future Rewards Based on Optimal Policy

Extension

- Recall our simple solution

$$Q^{new}(s,a) \leftarrow Q^{old}(s,a) + \alpha \cdot [\, r(s,a,s') - Q^{old}(s,a)] \,, \quad \text{with } \alpha \in (0,1)$$

- Consider the extension

$$Q^{new}(s,a) \leftarrow Q^{old}(s,a) + \alpha \cdot [\, \underbrace{r(s,a,s') + \gamma \cdot \max_{a'} Q^{old}(s',a') - Q^{old}\,(s,a)}_{\text{Non-Myopic Temporal Difference (TD)}}]$$

Learning Rule

- Based on simple logic and intuition, we have derived

$$Q^{new}(s,a) \leftarrow Q^{old}(s,a) + \alpha \cdot [\, r(s,a,s') + \gamma \cdot \max_{a'} Q^{old}(s',a') - Q^{old}\,(s,a)]$$

Learning Rule

- Based on simple logic and intuition, we have derived

$$Q^{new}(s,a) \leftarrow Q^{old}(s,a) + \alpha \cdot [\, r(s,a,s') + \gamma \cdot \max_{a'} Q^{old}(s',a') - Q^{old}\,(s,a)]$$

- May be expressed equivalently as

$$Q^{new}(s,a) \leftarrow (1-\alpha) \cdot Q^{old}(s,a) + \alpha \cdot [\, r(s,a,s') + \gamma \cdot \max_{a'} Q^{old}(s',a')\,]$$

Learning Rule

- Based on simple logic and intuition, we have derived

$$Q^{new}(s,a) \leftarrow Q^{old}(s,a) + \alpha \cdot [\, r(s,a,s') + \gamma \cdot \max_{a'} Q^{old}(s',a') - Q^{old}\,(s,a)]$$

- May be expressed equivalently as

$$Q^{new}(s,a) \leftarrow (1-\alpha) \cdot Q^{old}(s,a) + \alpha \cdot [\, r(s,a,s') + \gamma \cdot \max_{a'} Q^{old}(s',a')\,]$$

- Typically we let α diminish with time, and policy doesn't change substantially after seeing a lot of data, unless something very unusual occurs

Q Learning

$$Q^{new}(s,a) \leftarrow (1-\alpha) \cdot Q^{old}(s,a) + \alpha \cdot [\, r(s,a,s') + \gamma \cdot \max_{a'} Q^{old}(s',a') \,]$$

- We have derived an algorithm for a system/MD to learn based on experience

- This is actually a widely employed method for reinforcement learning, called Q Learning

Q Learning

$$Q^{new}(s,a) \leftarrow (1-\alpha)\cdot Q^{old}(s,a) + \alpha \cdot [\, r(s,a,s') + \gamma \cdot \max_{a'} Q^{old}(s',a') \,]$$

- We have derived an algorithm for a system/MD to learn based on experience

- This is actually a widely employed method for reinforcement learning, called Q Learning

- Based on the learned matrix $Q(s,a)$ the policy that is typically employed is

$$\pi(a;s) = \operatorname*{argmax}_{a} Q(s,a)$$

Q Learning

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

Q Learning

Discount factor, $\gamma \in [0,1]$

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

Q Learning

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

Learning rate, $\alpha \in [0,1]$

Q Learning

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot \underbrace{Q^{old}(s_t, a_t)}_{\text{Previous Estimate of Value of Action } a_t \text{ when in state } s_t} + \alpha \cdot [r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

Q Learning

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot \underbrace{[r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]}$$

Updated Estimate of Value of
Action a_t when in state s_t
After Observing r_t

Q Learning

Learning rate, $\alpha \in [0,1]$

Discount factor, $\gamma \in [0,1]$

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot \underbrace{Q^{old}(s_t, a_t)}_{\text{Previous Estimate of Value of Action } a_t \text{ when in state } s_t} + \alpha \cdot \underbrace{[r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]}_{\text{Updated Estimate of Value of Action } a_t \text{ when in state } s_t \text{ After Observing } r_t}$$

Q Learning

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

- Don't need to actually perform the action defined by $\operatorname{argmax}_a Q^{old}(s_{t+1}, a)$

Q Learning

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

- Don't need to actually perform the action defined by $\underset{a}{\operatorname{argmax}}\, Q^{old}(s_{t+1}, a)$

- Next action a_{t+1} may be based on $\underset{a_{t+1}}{\operatorname{argmax}}\, Q^{new}(s_{t+1}, a_{t+1})$, we then observe immediate reward r_{t+1} and new state s_{t+2}

$$\ldots\ s_t \quad a_t \quad r_t \quad s_{t+1} \quad a_{t+1} \quad r_{t+1} \quad s_{t+2}\ \ldots$$

Q Learning

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

- Don't need to actually perform the action defined by $\underset{a}{\operatorname{argmax}}\, Q^{old}(s_{t+1}, a)$

- Next action a_{t+1} may be based on $\underset{a_{t+1}}{\operatorname{argmax}}\, Q^{new}(s_{t+1}, a_{t+1})$, we then observe immediate reward r_{t+1} and new state s_{t+2}

$$\ldots\ s_t \quad a_t \quad r_t \quad s_{t+1} \quad a_{t+1} \quad r_{t+1} \quad s_{t+2}\ \ldots$$

- Sequentially and continually update the Q function $Q^{new}(s_t, a_t)$, which is an $n \times m$ matrix

Learning Rate

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

Learning rate, $\alpha \in [0,1]$

- Convergence guaranteed after sufficient experience, if learning rate diminishes with time
- In practice one may set $\alpha = 0.1$
- This online Q-learning setup allows one to learn an optimal policy based directly on experience

Q Learning vs. SARSA

- **Q-learning**, from previous slides:

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

Assumes access to s_t a_t r_t s_{t+1} (SARS); we don't implement a_{t+1} to update the Q functions

Q Learning vs. SARSA

- **Q-learning**, from previous slides:

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

Assumes access to $s_t\ a_t\ r_t\ s_{t+1}$ (SARS); we don't implement a_{t+1} to update the Q functions

- **SARSA** based learning considers the slightly modified

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r_t + \gamma \cdot Q^{old}(s_{t+1}, a_{t+1})]$$

Assumes access to $s_t\ a_t\ \ r_t\ \ s_{t+1}\ a_{t+1}$ (SARSA); a_{t+1} may be specified by some chosen mechanism

ϵ-Greedy: Exploration and Exploitation

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r_t + \gamma \cdot Q^{old}(s_{t+1}, a_{t+1})]$$

- Assume we observe $s_t \;\; a_t \;\; r_t \;\; s_{t+1}$

ϵ-Greedy: Exploration and Exploitation

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r_t + \gamma \cdot Q^{old}(s_{t+1}, a_{t+1})]$$

- Assume we observe $s_t \;\; a_t \;\; r_t \;\; s_{t+1}$

- Which next action a_{t+1} should we take within Q learning or SARSA?

ϵ-Greedy: Exploration and Exploitation

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r_t + \gamma \cdot Q^{old}(s_{t+1}, a_{t+1})]$$

- Assume we observe $s_t \;\; a_t \;\; r_t \;\; s_{t+1}$

- Which next action a_{t+1} should we take within Q learning or SARSA?

- ϵ-Greedy (small ϵ):

  - With probability ϵ, choose a_{t+1} at random
  - With probability $1-\epsilon$, next action $a_{t+1} = \underset{a}{\operatorname{argmax}}\, Q^{old}(s_{t+1}, a)$

ϵ-Greedy: Exploration and Exploitation

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r_t + \gamma \cdot Q^{old}(s_{t+1}, a_{t+1})]$$

- Assume we observe $s_t \;\; a_t \;\; r_t \;\; s_{t+1}$

- Which next action a_{t+1} should we take within Q learning or SARSA?

- ϵ-Greedy (small ϵ):

  - With probability ϵ, choose a_{t+1} at random
  - With probability $1-\epsilon$, next action $a_{t+1} = \underset{a}{\operatorname{argmax}}\, Q^{old}(s_{t+1}, a)$

Allows exploration with probability ϵ

Further Insights on Q-Learning

- Recall that Q learning is characterized by the update equation

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r(s_t, a_t, s_{t+1}) + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

Further Insights on Q-Learning

- Recall that Q learning is characterized by the update equation

$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r(s_t, a_t, s_{t+1}) + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

- For simplicity, consider the case $\alpha = 1$ and hence the update

$$Q^{new}(s_t, a_t) \leftarrow r(s_t, a_t, s_{t+1}) + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)$$

Further Insights on Q-Learning

$$Q^{new}(s_t, a_t) \leftarrow r(s_t, a_t, s_{t+1}) + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)$$

Further Insights on Q-Learning

$$Q^{new}(s_t, a_t) \leftarrow r(s_t, a_t, s_{t+1}) + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)$$

- For all actions $a \in A$ we similarly have

$$Q^{old}(s_{t+1}, a_{t+1}) \leftarrow r(s_{t+1}, a_{t+1}, s_{t+2}) + \gamma \cdot \max_a Q^{older}(s_{t+2}, a)$$

Repeated Use of Update Equation

$$Q^{new}(s_t, a_t) \leftarrow r(s_t, a_t, s_{t+1}) + \gamma \cdot \max_a \boxed{Q^{old}(s_{t+1}, a)}$$

- For all actions $a \in A$ we similarly have

$$\boxed{Q^{old}(s_{t+1}, a_{t+1})} \leftarrow r(s_{t+1}, a_{t+1}, s_{t+2}) + \gamma \cdot \max_a Q^{older}(s_{t+2}, a)$$

Looking Two Steps Ahead

- Using the expression for $Q^{old}\ (s_{t+1}, a)$, we therefore have

$$Q^{new}(s_t, a_t) \leftarrow r(s_t, a, s_{t+1}) + \max_a \underbrace{[\,\gamma \cdot r(s_{t+1}, a, s_{t+2}) + \gamma^2 \cdot \max_{a'} Q^{older}(s_{t+2}, a')]}_{= \gamma \cdot Q^{old}\ (s_{t+1}, a)}$$

Result of Sequence of Iterations

- Repeating this process we now have

$$Q^{new}(s_t, a_t) \leftarrow r(s_t, a, s_{t+1}) + \max_a [\, \gamma \cdot r(s_{t+1}, a, s_{t+2}) + \gamma^2 \cdot \max_{a'} Q^{older}(s_{t+2}, a')]$$

Result of Sequence of Iterations

- Repeating this process we now have

$$Q^{new}(s_t, a_t) \leftarrow r(s_t, a, s_{t+1}) + \max_a [\, \gamma \cdot r(s_{t+1}, a, s_{t+2}) + \gamma^2 \cdot \max_{a'} Q^{older}(s_{t+2}, a')]$$

$$\leftarrow r(s_t, a, s_{t+1}) + \max_a \{\, \gamma \cdot r(s_{t+1}, a, s_{t+2}) + \max_{a'} [\gamma^2 \cdot r(s_{t+2}, a', s_{t+3}) + \ldots$$

Result of Sequence of Iterations

- Repeating this process we now have

$$Q^{new}(s_t, a_t) \leftarrow r(s_t, a, s_{t+1}) + \max_a [\, \gamma \cdot r(s_{t+1}, a, s_{t+2}) + \gamma^2 \cdot \max_{a'} Q^{older}(s_{t+2}, a')]$$

$$\leftarrow r(s_t, a, s_{t+1}) + \max_a \{\, \gamma \cdot r(s_{t+1}, a, s_{t+2}) + \max_{a'} [\gamma^2 \cdot r(s_{t+2}, a', s_{t+3}) + \ldots$$

Reward scales by multiplicative factor γ^t for t steps into the future

Result of Sequence of Iterations

- We therefore have

$$Q^{new}(s_t, a_t) \leftarrow r(s_t, a, s_{t+1}) + \max_a [\, \gamma \cdot r(s_{t+1}, a, s_{t+2}) + \gamma^2 \cdot \max_{a'} Q^{older}(s_{t+2}, a')]$$

$$\leftarrow r(s_t, a, s_{t+1}) + \max_a \{\, \gamma \cdot r(s_{t+1}, a, s_{t+2}) + \max_{a'} [\gamma^2 \cdot r(s_{t+2}, a', s_{t+3}) + \ldots$$

Implement the max operations from horizon to the present time (right to left)

Termed a “backup algorithm”

Meaning of Q Function

- With sufficient training data (experience), the Q function represents

$$Q(s,a) = Average\left[r(s,a,s_{t=1}) + \underbrace{\sum_{t=1}^{\infty} \gamma^t r(s_t,a_t,s_{t+1})}_{\text{Actions based on optimal policy } \pi(a;s)}\right]$$

- Average taken over environment statistics $P(s,a,s')$, which are not known explicitly, but experienced when learning

Meaning of Q Function

- With sufficient training data (experience), the Q function represents

$$Q(s,a) = Average\left[r(s,a,s_{t=1}) + \underbrace{\sum_{t=1}^{\infty} \gamma^t r(s_t,a_t,s_{t+1})}_{\text{Actions based on optimal policy } \pi(a;s)}\right]$$

- Average taken over environment statistics $P(s,a,s')$, which are not known explicitly, but experienced when learning

- Mathematically, the aforementioned average is termed an "expectation" wrt $P(s,a,s')$

Limitations of Tabular Q Learning

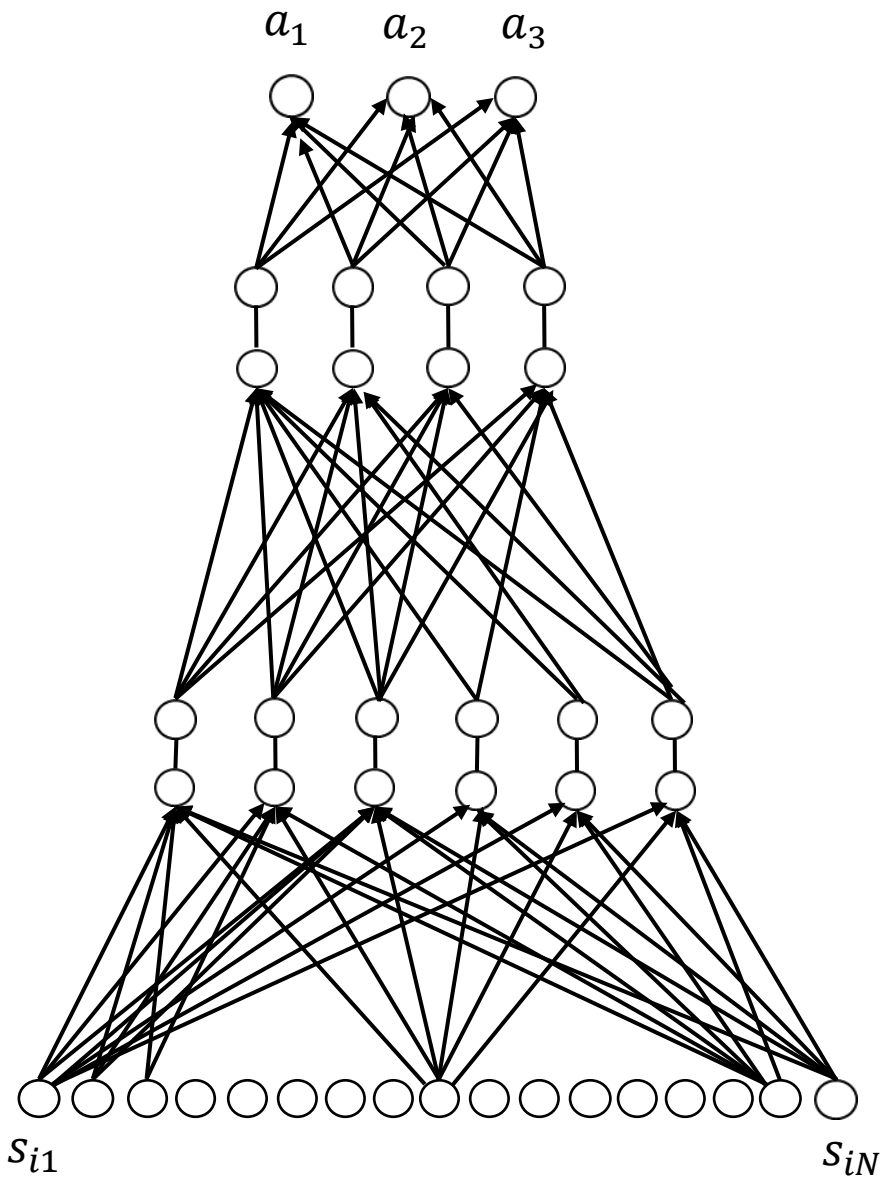
$$Q^{new}(s_t, a_t) \leftarrow (1-\alpha) \cdot Q^{old}(s_t, a_t) + \alpha \cdot [r_t + \gamma \cdot \max_a Q^{old}(s_{t+1}, a)]$$

- Simple update rule, but …..

  - The Q function is stored as a table/matrix, which is impractical when considering a large number of states and actions

  - No real capacity to generalize across sequence types, because the tabular Q function does not have a functional form

Represent Q Function as a Neural Network



- The Q-function modeled via a neural network
- The state is input (e.g., an image with *N* pixels), and at the output we model the Q-function $Q(s, a)$ for each possible action
- Represent the neural network model as $Q(s, a; \theta)$ where θ represent the neural network parameters we wish to learn
- After we learn θ the policy is

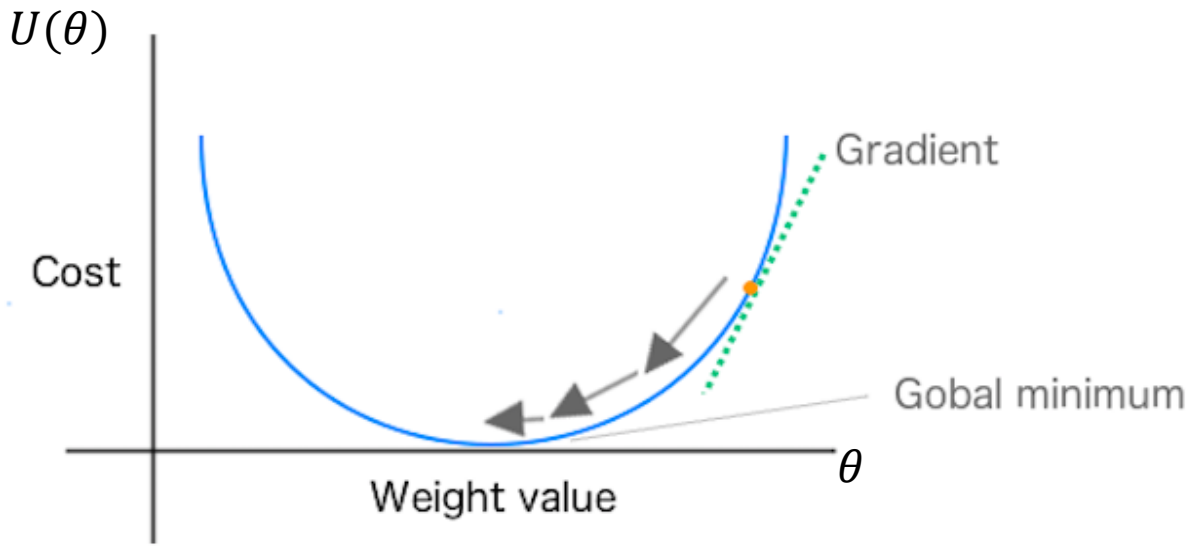
$$\pi(s) = \underset{a}{\operatorname{argmax}}\, Q(s, a; \theta)$$

Deep Q Learning (DQN)

- If we update θ like in Q learning, with every new observed $s_t \;\; a_t \;\; r_t \;\; s_{t+1}$, at each step we seek to minimize

$$U(\theta;\ s_t, a_t) = \frac{1}{2}[Q(s_t, a_t; \theta) - [r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^{old})]\ ]^2$$

- To move θ such that $U(\theta; s_t, a_t)$ is made smaller, we update model parameters as



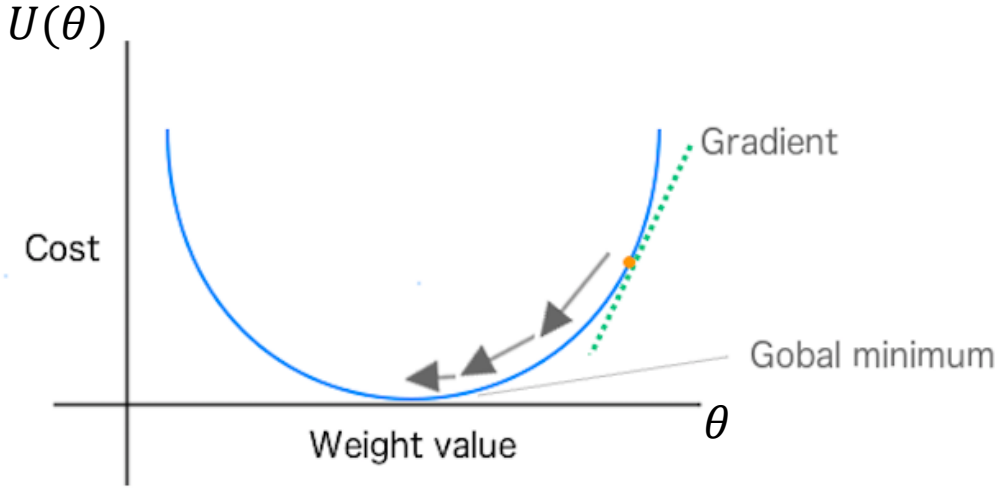
Gradient Descent

- If we update θ like in Q learning, with every new observed $s_t \;\; a_t \;\; r_t \;\; s_{t+1}$, at each step we seek to minimize

$$U(\theta;\ s_t, a_t) = \frac{1}{2}[Q(s_t, a_t; \theta) - [r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^{old})]\,]^2$$

- To move θ such that $U(\theta; s_t, a_t)$ is made smaller, we update model parameters as

$$\theta^{new} \leftarrow \theta^{old} - \alpha \cdot \nabla_\theta U(\theta; s_t, a_t)|_{\theta=\theta^{old}}$$



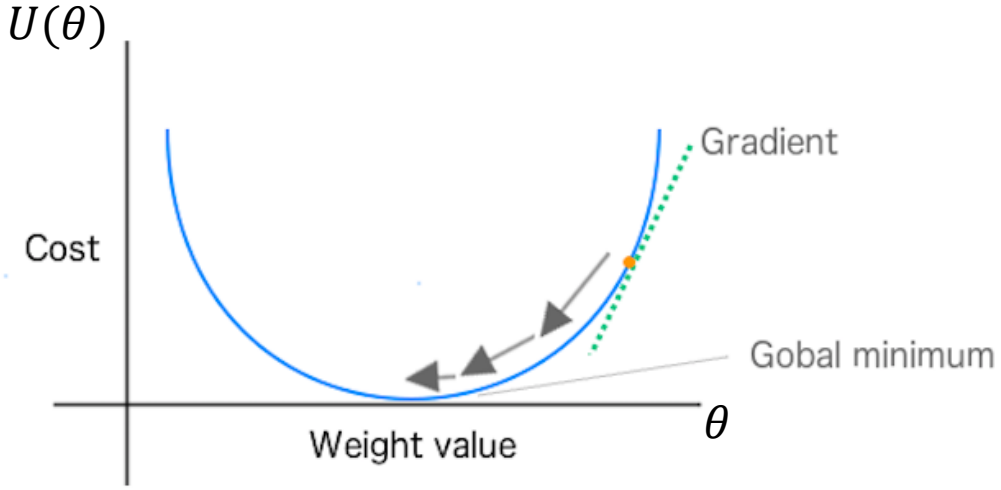
Gradient Descent

- If we update θ like in Q learning, with every new observed $s_t \;\; a_t \;\; r_t \;\; s_{t+1}$, at each step we seek to minimize

$$U(\theta;\ s_t, a_t) = \frac{1}{2}[Q(s_t, a_t; \theta) - [r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^{old})]\,]^2$$

- To move θ such that $U(\theta; s_t, a_t)$ is made smaller, we update model parameters as

$$\theta^{new} \leftarrow \theta^{old} - \alpha \cdot \underbrace{\nabla_\theta U(\theta; s_t, a_t)|_{\theta=\theta^{old}}}_{\text{Multi-dimensional “slope”}}$$



Gradient Descent

- If we update θ like in Q learning, with every new observed $s_t \;\; a_t \;\; r_t \;\; s_{t+1}$, at each step we seek to minimize

$$U(\theta;\ s_t, a_t) = \frac{1}{2}[Q(s_t, a_t; \theta) - [r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^{old})]\ ]^2$$

- To move θ such that $U(\theta; s_t, a_t)$ is made smaller, we update model parameters as

$$\theta^{new} \leftarrow \theta^{old} - \alpha \cdot \nabla_\theta U(\theta; s_t, a_t)_{|_{\theta=\theta^{old}}}$$

$$\theta^{new} \leftarrow \theta^{old} - \alpha \cdot \left[Q\left(s_t, a_t; \theta^{old}\right) - r_t - \gamma \cdot \max_{a'} Q\left(s_{t+1}, a'; \theta^{old}\right)\right] \nabla_\theta \mathrm{Q}(s_t, a_t; \theta)_{|_{\theta=\theta^{old}}}$$

Gradient Descent

$$\theta^{new} \leftarrow \theta^{old} + \alpha \cdot \left[r_t + \gamma \cdot \max_{a'} Q\left(s_{t+1}, a'; \theta^{old}\right) - Q(s_t, a_t; \theta^{old})\right] \nabla_\theta \mathrm{Q}(s_t, a_t; \theta)_{|_{\theta=\theta^{old}}}$$

Looks a lot like the prior Q learning

Gradient Descent

$$\theta^{new} \leftarrow \theta^{old} + \alpha \cdot \left[r_t + \gamma \cdot \max_{a'} Q\left(s_{t+1}, a'; \theta^{old}\right) - Q(s_t, a_t; \theta^{old}) \right] \underbrace{\nabla_\theta \mathrm{Q}(s_t, a_t; \theta)_{|_{\theta=\theta^{old}}}}$$

Modification to account for neural network parameters θ

Gradient Descent

$$\theta^{new} \leftarrow \underbrace{\theta^{old} + \alpha \cdot \left[r_t + \gamma \cdot \max_{a'} Q\left(s_{t+1}, a'; \theta^{old}\right) - Q(s_t, a_t; \theta^{old})\right]}_{\text{Looks a lot like the prior Q learning}} \underbrace{\nabla_\theta \mathrm{Q}(s_t, a_t; \theta)_{|_{\theta=\theta^{old}}}}_{\text{Modification to account for neural network parameters } \theta}$$

- One may show rigorously that this is directly related to the prior sequential Q learning

- Details not very difficult (first-order Taylor series expansion), but beyond the scope of our discussion

Revisit What Deep Q Network Seeks to Learn

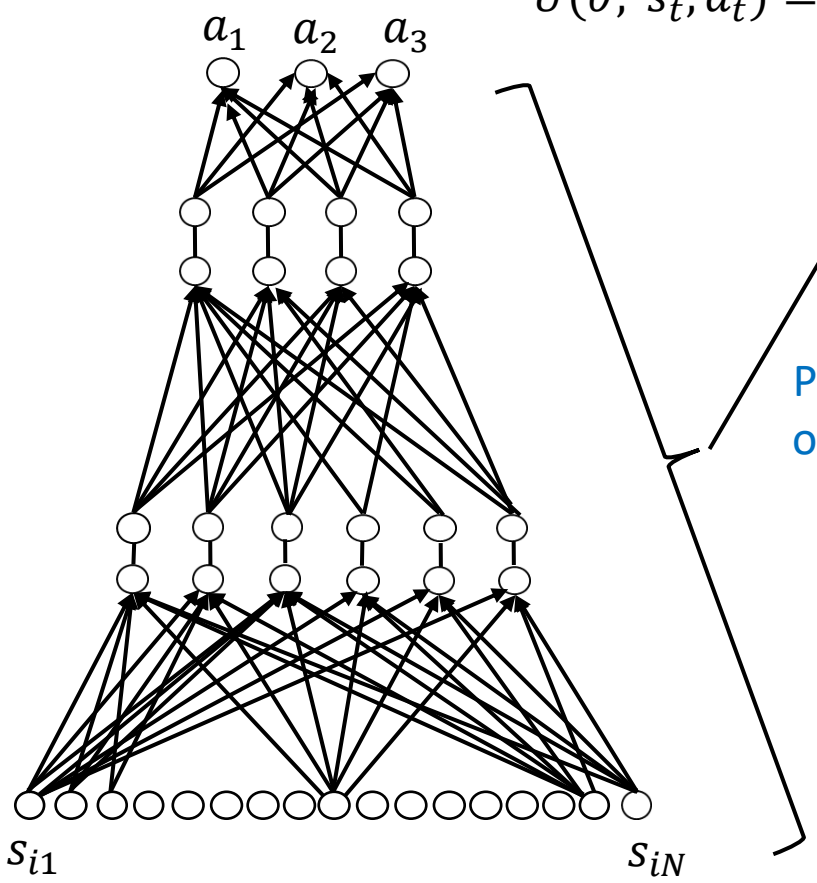
- Seek parameters θ that minimize prediction error

$$U(\theta;\ s_t, a_t) = \frac{1}{2}\left[Q(s_t, a_t; \theta) - \left[r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^{old})\right]\right]^2$$

Revisit What Deep Q Network Seeks to Learn

- Seek parameters θ that minimize prediction error

$$U(\theta;\ s_t, a_t) = \frac{1}{2}[Q(s_t, a_t; \theta) - [r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^{old})]\ ]^2$$



Predictive model for value of state-action pair (s_t, a_t)

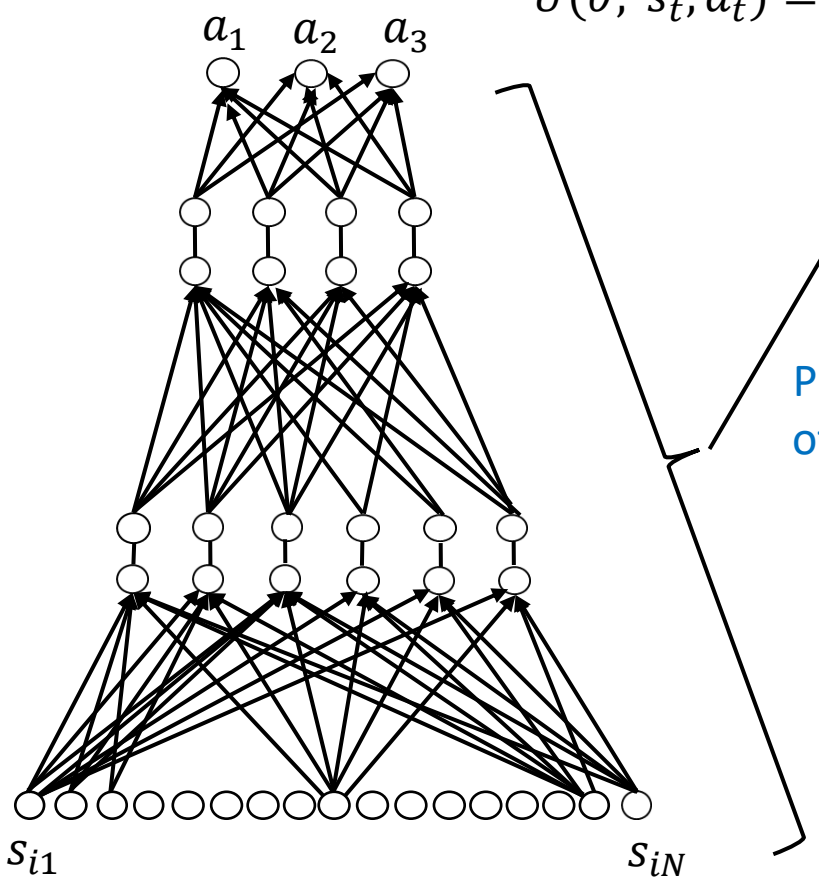
Revisit What Deep Q Network Seeks to Learn

- Seek parameters θ that minimize prediction error

Discounted future rewards

$$U(\theta;\ s_t, a_t) = \frac{1}{2}[Q(s_t, a_t; \theta) - [r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^{old})]\ ]^2$$

Next immediate reward



Predictive model for value of state-action pair (s_t, a_t)

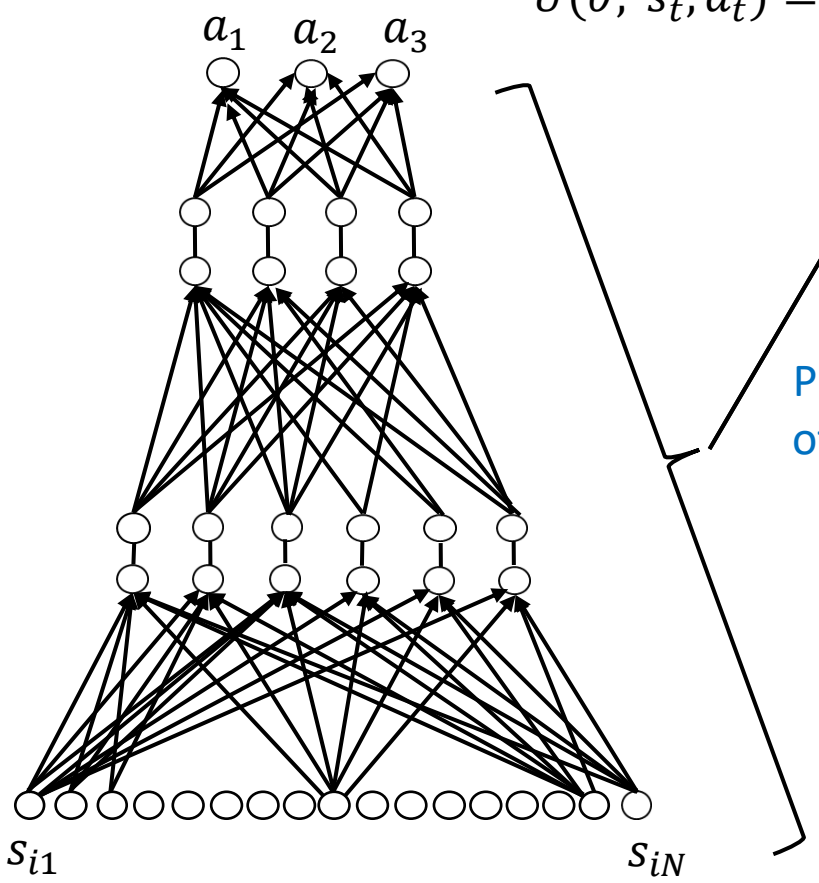
Revisit What Deep Q Network Seeks to Learn

- Seek parameters θ that minimize prediction error

Discounted future rewards

$$U(\theta;\ s_t, a_t) = \frac{1}{2}[Q(s_t, a_t; \theta) - [r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^{old})]\ ]^2$$

Next immediate reward



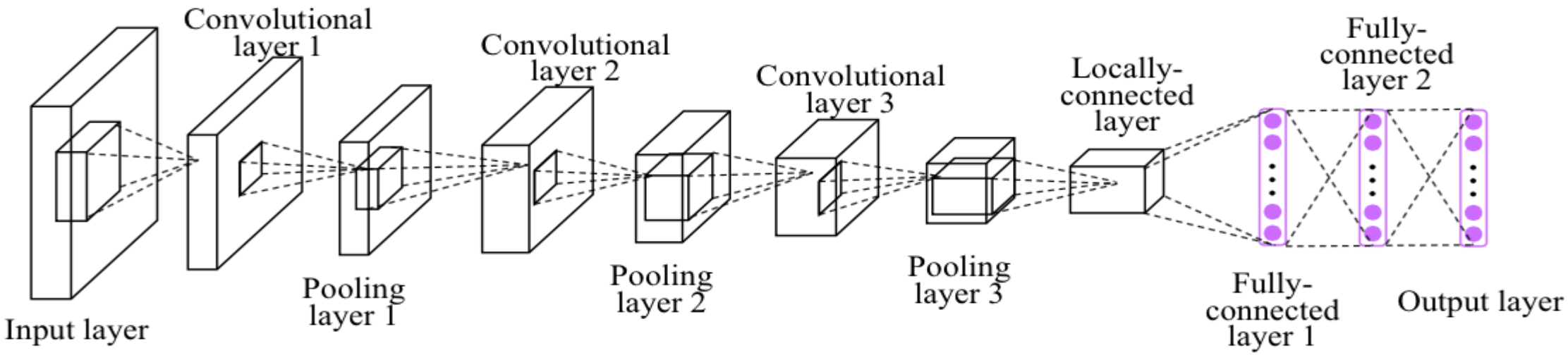
Predictive model for value of state-action pair (s_t, a_t)

Supervised Learning

Power of Deep Q Networks for RL With Images

- Seek parameters θ that minimize prediction error

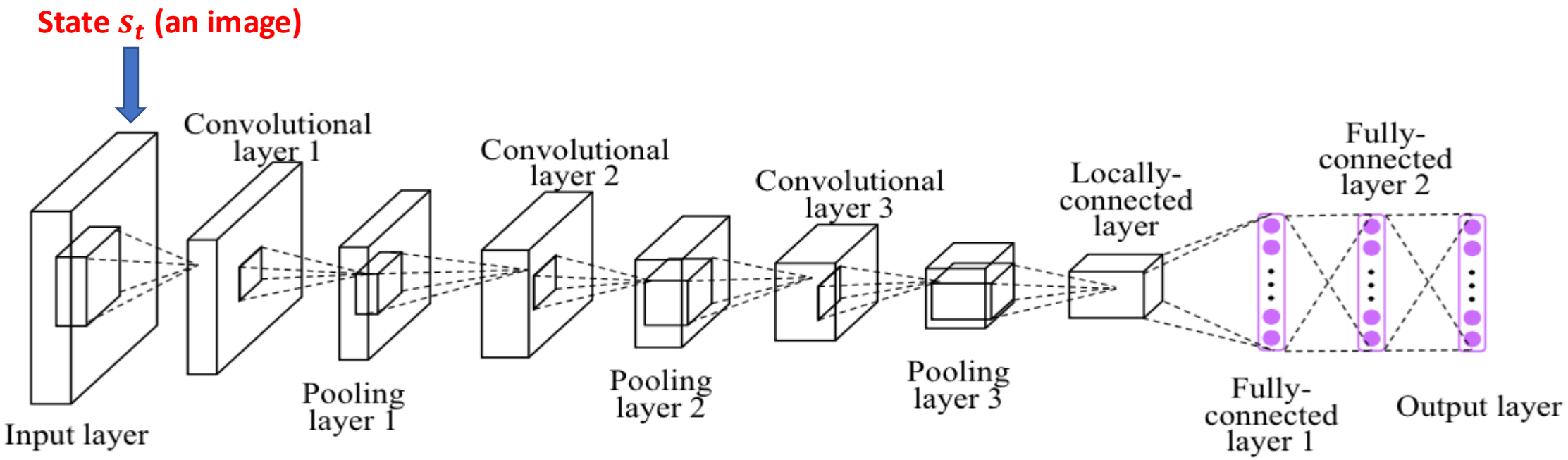
$$U(\theta;\ s_t, a_t) = \frac{1}{2}[Q(s_t, a_t; \theta) - [r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^{old})]\ ]^2$$



Power of Deep Q Networks for RL With Images

- Seek parameters θ that minimize prediction error

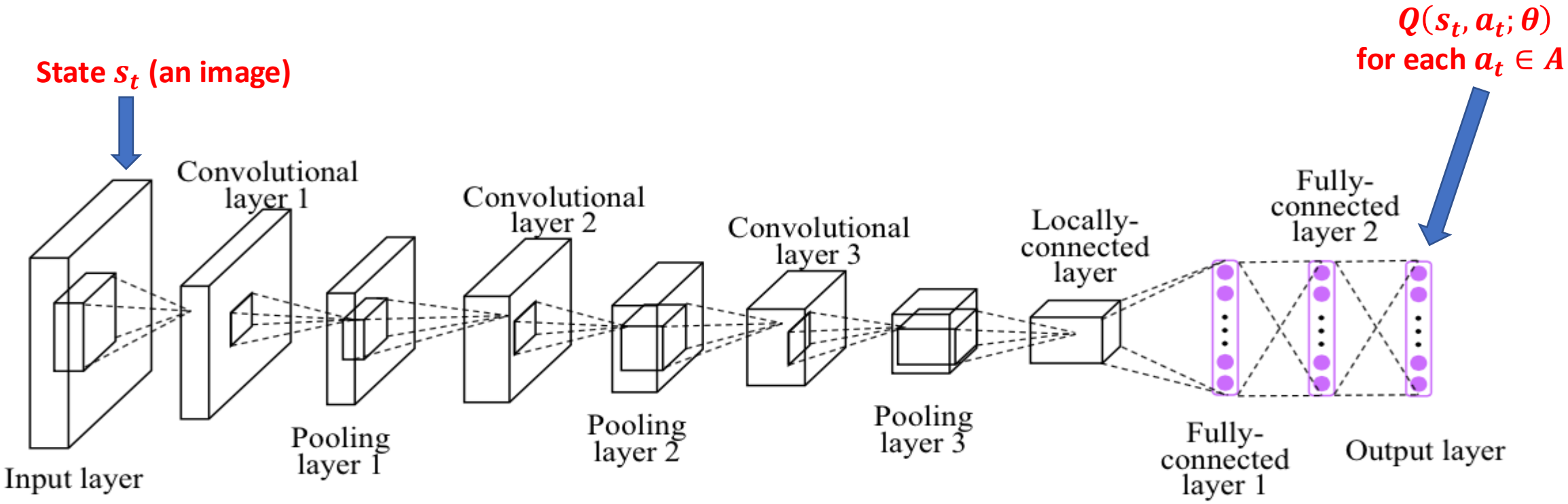
$$U(\theta;\ s_t, a_t) = \frac{1}{2}[Q(s_t, a_t; \theta) - [r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^{old})]\ ]^2$$



Power of Deep Q Networks for RL With Images

- Seek parameters θ that minimize prediction error

$$U(\theta;\ s_t, a_t) = \frac{1}{2}[Q(s_t, a_t; \theta) - [r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^{old})]\ ]^2$$



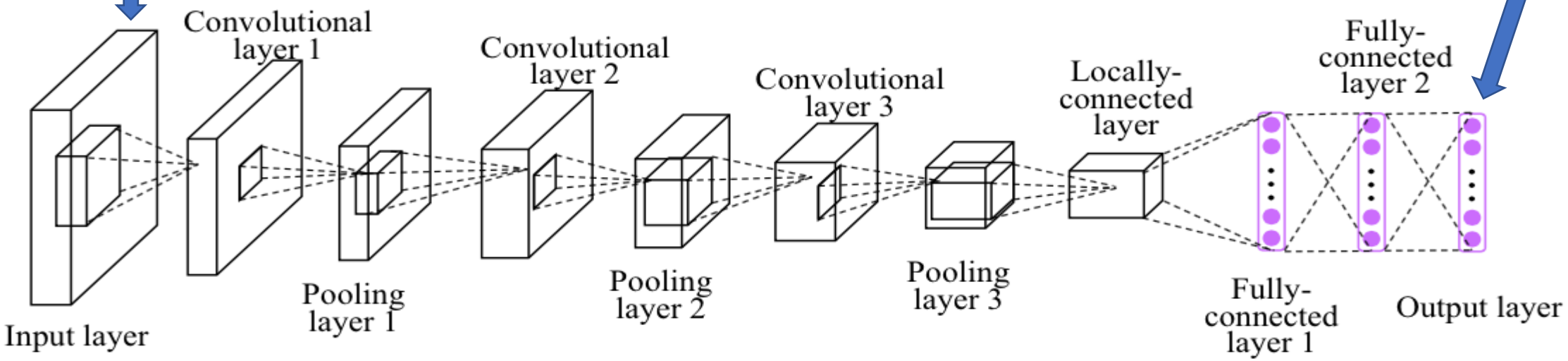
Power of Deep Q Networks for RL With Images

- Seek parameters θ that minimize prediction error

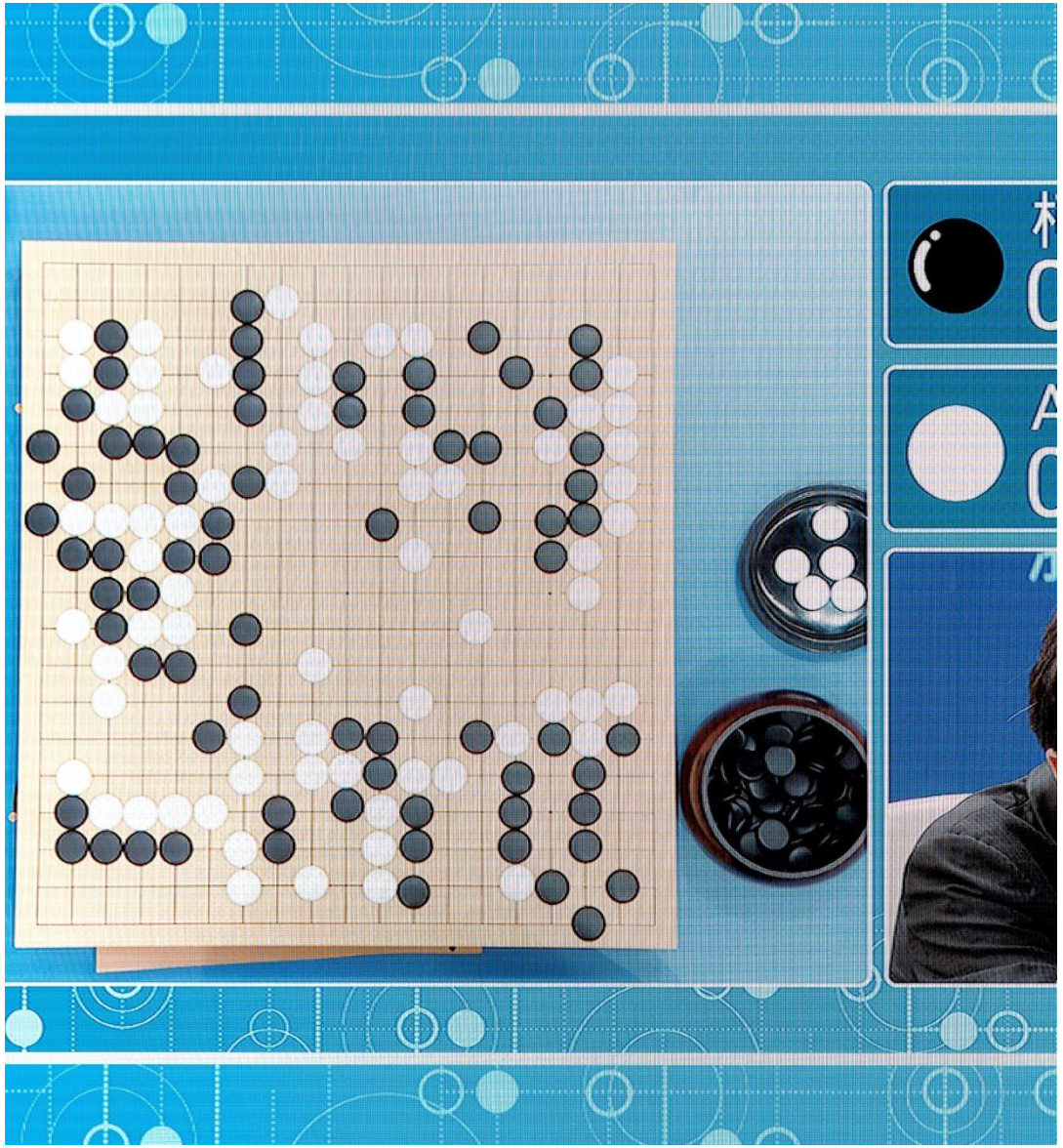
$$U(\theta;\ s_t, a_t) = \frac{1}{2}[Q(s_t, a_t; \theta) - [r_t + \gamma \cdot \max_{a'} Q(s_{t+1}, a'; \theta^{old})]\ ]^2$$

$\boldsymbol{Q(s_t, a_t; \theta)}$
for each $\boldsymbol{a_t \in A}$

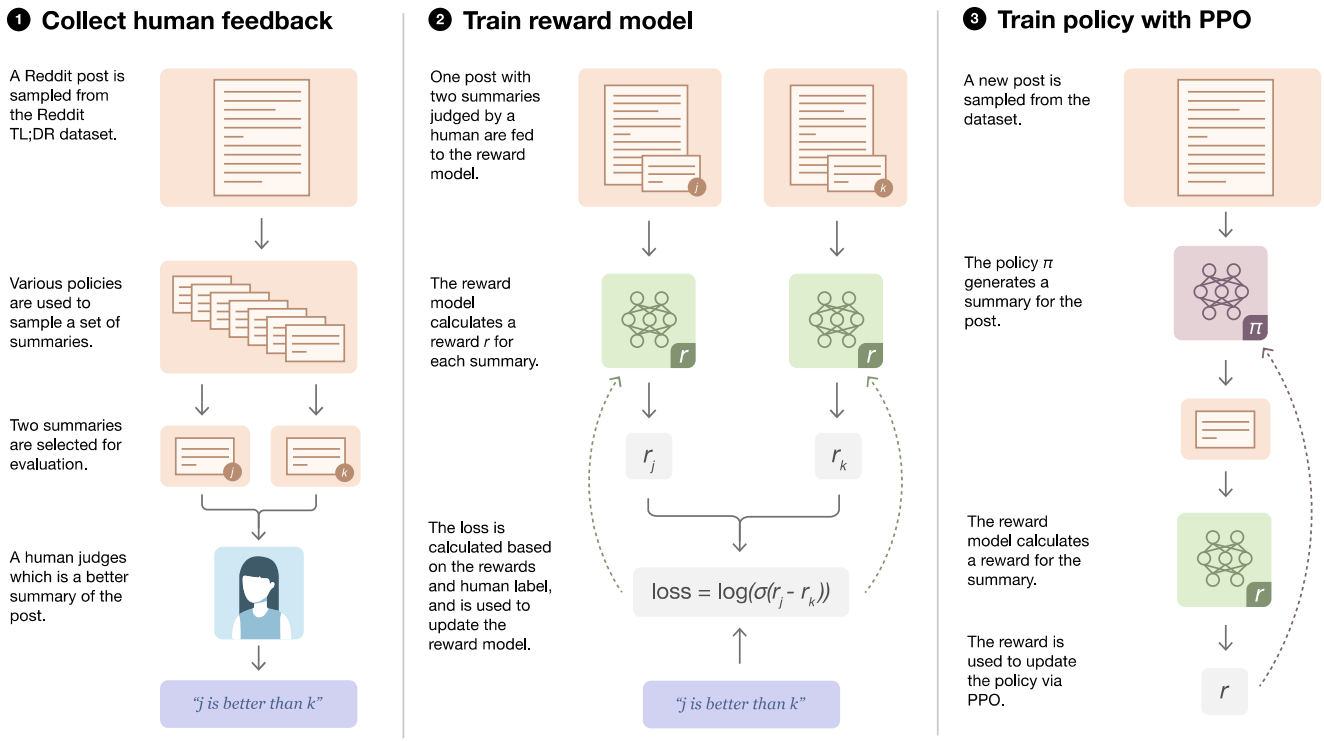
State $\boldsymbol{s_t}$ (an image)



Google DeepMind
Challenge Match
8 - 15 March 2016
ALPHAGO
01:59:54
LEE SEDOL
01:59:02
AlphaGo



Atari Games

Extend RL for Language Modeling

- How do we extend RL ideas to the characteristics of language models?
- Would like to encourage LLM to generate text that is aligned with the desires of people
- Would like to encourage certain behaviors of the LLM: chain-of-thought reasoning